

HA-CAAP: Hardware-Assisted Continuous Authentication and Attestation Protocol for IoT Based on Blockchain

Vangelis Malamas[✉], *Member, IEEE*, Panayiotis Kotzanikolaou[✉], *Member, IEEE*, Kostas Nomikos, Christos Zonios[✉], Vasileios Tenentes[✉], *Member, IEEE*, and Mihalis Psarakis[✉], *Member, IEEE*

Abstract—The increasing integration of Internet of Things (IoT) devices in various sectors has created complex and dynamically changing interconnected systems. In several multiauthority and multidomain applications, IoT devices may continuously change their connectivity status, leading to dynamic topologies; an IoT device may be connected to different gateways at different times, to support the provisioning of a distributed service. However, these complex environments increase exposure to security threats, such as device spoofing or cloning attacks. Even worse, without continuous device inventorying, an adversary may easily duplicate a cloned device and concurrently connect compromised Sybil nodes at different gateways, without getting noticed. This article proposes Hardware-assisted, continuous authentication and attestation protocol (HA-CAAP), a hardware-assisted continuous authentication and attestation protocol for IoT devices. By using a physically unclonable function-based periodic authentication mechanism, gateways can continuously authenticate their connected devices and detect modifications in their connectivity status, in nearly real-time. Through a private blockchain, gateways are able to continuously exchange information about their connectivity state and securely share a dynamic device inventory, to detect possible Sybil attacks. In addition, by integrating a continuous gateway attestation mechanism in the blockchain, the protocol prevents nontrusted gateways from joining in and assures their integrity. To evaluate HA-CAAP, security is formally analyzed, while a proof-of-concept implementation is used to analyze the protocol's performance, for realistic application scenarios.

Index Terms—Continuous attestation, continuous authentication, hardware-assisted security, Internet of Things (IoT), key agreement.

I. INTRODUCTION

THE Internet of Things (IoT) technology enables the interconnectivity of edge devices with backend computing systems in various domains, such as healthcare, manufacturing, and supply chain management. However, the widespread

adoption of IoT has also led to increased exposure to various security threats. Since IoT devices usually operate in dynamic, multiauthority and multidomain environments, ensuring protection against security attacks, such as data leakage, device spoofing, or cloning attacks, is not a trivial task.

Nowadays, computing devices may be equipped with hardware Root of Trust (RoT) mechanisms, which rely on tamper-resistant hardware to ensure system integrity. RoT is immutable and resistant to software-based attacks, making it a trusted base for critical functions like device authentication and key protection. Two key RoT primitives are trusted execution environments (TEEs) and physically unclonable functions (PUFs). TEEs create secure enclaves that support secure boot, firmware integrity, and remote attestation. PUFs are an RoT alternative suitable for low-capability devices, such as IoT. They use a device's unique physical characteristics to support authentication and cryptographic key generation. However, although PUFs strengthen device resistance against spoofing and cloning attacks, they are not considered as secure as TEE.

Although PUF-based authentication may protect against remote spoofing and cloning attacks, there are still open challenges, especially in dynamic multiauthority and multidomain environments. Since IoT devices authenticate to gateways at sparse periods (usually during login), attackers with physical access may find sufficient time to physically tamper devices. An adversary may also perform Sybil attacks i.e., clone a compromised device and use multiple clones to concurrently connect to gateways at different network domains. Combining Sybil and cloning attacks may significantly amplify the attack impact. For example, concurrently bypassing authentication, exfiltrating data, or connectivity state on several domains. Due to the lack of central device connectivity status management, the Sybil nodes can be concurrently connected to different domains for a long time, before being detected. In dynamic environments where IoT nodes may change network domains, such attacks are hard to detect in real-time [1].

A. Motivating Application Scenarios

IoT spoofing and Sybil attacks are common threats in various multiauthority and multidomain environments. Typical examples are as follows.

Received 3 October 2024; revised 23 December 2024; accepted 8 January 2025. Date of publication 16 January 2025; date of current version 23 May 2025. This work was supported in part by the University of Piraeus Research Centre, and in part by HEAL-Link. (Corresponding author: Panayiotis Kotzanikolaou.)

Vangelis Malamas, Panayiotis Kotzanikolaou, Kostas Nomikos, and Mihalis Psarakis are with the Department of Informatics, University of Piraeus, 185 34 Piraeus, Greece (e-mail: bagmalamas@unipi.gr; pkotzani@unipi.gr; knomik@unipi.gr; mpsarak@unipi.gr).

Christos Zonios and Vasileios Tenentes are with the Department of Computer Science and Engineering, University of Ioannina, 451 10 Ioannina, Greece (e-mail: c.zonios@uoi.gr; tenentes@uoi.gr).

Digital Object Identifier 10.1109/JIOT.2025.3530775

Real-Time and Continuous Authentication of Medical Devices: Internet of Medical Things (IoMT) devices (e.g., sensors, pacemakers, and infusion pumps) are often distributed across various physical areas, networks and gateways, and may be associated with different patients at different times. Real security incidents have been reported in hospitals [1], wherein unauthorized medical IoT devices have bypassed authentication controls and infiltrated hospital networks. In addition, compromised cloned devices have been used to connect to trusted gateways without getting noticed. Consequently, hospital administrators must be able to detect cloned devices in real-time. Furthermore, as devices dynamically change location, there is a need for a secure real-time inventorying mechanism, in order to have a clear view of all devices' current connection status and to prevent Sybil attacks.

Real-Time and Continuous Authentication of Supply Chain Tracking Devices: In the supply chain industry, IoT devices facilitate real-time asset tracking, monitoring, and alerting. Sensors placed on products or containers may connect to different gateways belonging to different authorities (e.g., producers, transporters, or sale points), to transmit product tracking and monitoring data. The integrity and confidentiality of the information exchanged between sensors and gateways may be critical. For instance, in a food supply chain, the trustworthiness of the devices and the integrity of the collected data are vital to ensure that the products meet the required quality specifications and remain unaltered. Thus, there is a need to continuously authenticate IoT devices and to ensure secure communications between gateways and IoT devices, despite the frequent connectivity changes.

B. Research Gaps

Existing continuous IoT authentication mechanisms are targeted to application environments where the IoT devices and the gateways are managed by the same authority, which simplifies trust management, real-time inventorying and centralized Sybil attack detection (e.g., [2], [3], [4], [5], [6], [7], and [8]). However, in multiauthority environments where devices may connect to gateways operated by different authorities at different times, there is a need for continuous IoT authentication which supports: 1) distributed trust management, to allow interoperability of devices and gateways managed by different authorities; 2) synchronization and real-time inventorying for Sybil attack prevention in multiple network domains; and 3) strong guarantees for the integrity of the distributed protocol execution.

C. Contribution

We propose, a Hardware-Assisted Continuous Authentication and Attestation Protocol (HA-CAAP). On the IoT side, HA-CAAP employs PUF-based continuous authentication, which prevents remote spoofing adversaries, and limits their exposure only to spoofing adversaries with physical tampering capabilities. On the gateway side, HA-CAAP is based on the fusion of hardware- and software-based immutability mechanisms. A TEE-based continuous attestation mechanism is proposed that continuously assures

the trustworthiness of the gateways in a distributed manner. In addition, a private blockchain with smart contracts executed by the gateways, allows for secure real-time exchange of IoT connectivity information and synchronization. In summary:

- 1) HA-CAAP provides PUF-assisted continuous authentication for IoT devices. It enables gateways to authenticate and to *continuously detect connectivity changes* of connected IoT devices in real-time. The protocol is agnostic of the underlying PUF depending on the PUF strength (strong or weak), additional shared secrets may be used, to increase entropy and to retain the required security level;
- 2) HA-CAAP allows gateways to establish *pairwise keys* for secure unicast communication with IoT devices, and *group keys* that may be used for broadcasting. All keys are *dynamically updated* upon detection of a connectivity change;
- 3) in addition, HA-CAAP's *gateway attestation mechanism*, allows gateways to continuously attest each other, e.g., prove the authenticity and integrity of their execution state. Continuous attestation combines TEE and blockchain modules, both running on the gateways' side;
- 4) the integration of a private blockchain at the gateways allows HA-CAAP to support *real-time synchronization in distributed, multiauthority environments*. In addition, it enables gateways to run part of the protocol functionality via smart contracts. Blockchain immutability assures the integrity of those parts of the protocol executed outside the gateways' TEE. Watchdogs running as smart contracts assure real-time protection from race conditions that may derive due to the distributed protocol execution. In this way, gateways continuously and securely exchange information regarding: 1) the *connectivity state of IoT devices*, to detect Sybil attacks and 2) the *attestation state* of other gateways; and
- 5) finally, we have implemented and measured HA-CAAP performance in terms of energy consumption and execution time in realistic application scenarios to show that it can be executed in constrained IoT computing devices.

D. Paper Structure

In Section II, we review existing authentication schemes for IoT devices. In Section III, we present HA-CAAP, while in Section IV we formally analyze its security. In Section V, we present a testbed implementation, which is then utilized to examine the performance and efficiency of the protocol for realistic application scenarios. Finally, Section VI concludes this article.

II. BACKGROUND AND RELATED WORK

In this section, we provide an overview of hardware RoT key technologies relevant to this work. We then review related works that propose hardware-assisted and blockchain-based protocols, examining their strengths and weaknesses in addressing continuous authentication, Sybil attack protection, and secure communication in distributed IoT networks.

A. Background

Hardware RoT establishes a trusted foundation in hardware for system integrity. It uses tamper-resistant hardware modules, which are immutable and resist software-based attacks, to ensure device authenticity, critical function integrity, and key protection. Typical RoT primitives include TEEs and PUFs.

TEEs are secure enclave extensions that offer services, such as strong device authentication, secure booting, software integrity, and remote attestation [9]. They rely on secure modules that store immutable key pairs, with public keys cryptographically linked to the hardware via vendor-signed certificates. This enables a chain of trust that starts at boot time and continues at run-time. Remote attestation involves signing messages with trusted module keys for verification by a centralized trusted authority, often the vendor. However, centralized attestation may limit trust and scalability. Decentralized solutions, especially in IoT networks, could improve security, interoperability, and transparency while reducing costs. Major processor vendors like ARM, Intel, and AMD offer integrated TEE hardware primitives [10], [11], [12], such as the ARM TrustZone (mainly used on mobile devices and IoT gateways), Intel SGX (used in workstations), and AMD SEV (used by Linux KVM virtual machines), but their high cost limits their use in IoT devices.

PUF is an RoT primitive that relies on the unique characteristics of a device, which stem from the variability of the manufacturing process, for the generation of fingerprints and keys. PUFs are used to design hardware-assisted authentication protocols; a challenger (e.g., a gateway) having access to a fingerprint corresponding to the device's physical characteristic(s) may challenge the device to authenticate by utilizing those characteristics to generate a valid response.

PUFs come in two main categories. *Strong PUF* are higher-cost constructions that can be used both for device authentication and cryptographic key generation. As with TEE, strong PUF may not be viable for IoT devices in all application domains. In such cases, *weak PUF* can be applied. These have lower deployment and energy consumption, but at the same time, have weaker security level. However, when combined with additional sources of randomness, such as shared secret keys, weak PUFs may also support strong device authentication with hardware security guarantees, i.e., cloning or spoofing attacks will require physical tampering of the device [13], [14], [15].

B. Related Work

In the literature, various hardware-assisted IoT authentication protocols have been proposed (e.g., [16], [17], [18], and [19]), which, however, are not designed for continuous authentication and, therefore, cannot support real-time inventorying and Sybil attack protection in multiauthority environments.

Given that IoT devices are usually the weakest link in the security chain and often serve as the initial point of entry for cyber attackers, several works have identified the need for continuous IoT authentication. In [2], a periodic continuous IoT authentication protocol is proposed. An authentication token is generated during a static authentication process and

is subsequently utilized during the continuous authentication phase. However, this protocol is not designed to resist Sybil attacks. Yu et al. [5] proposed a continuous authentication protocol based on Channel State Information. The security model of the protocol only analyzes ID-based attacks and has high computational cost [3]. In [20], a context-aware continuous authentication model is proposed that capitalizes on the observation that devices located at the same place consistently perceive similar ambient context information. Through the analysis of the context data, devices are continuously authenticated, assuming that the sensors used to infer context are trusted. In addition, all the above mentioned solutions maintain strong trust assumptions on third-party servers, which are actively involved during the authentication process.

Recent studies try to reduce trust assumptions by using blockchain to share authentication information among devices. In [21], a continuous authentication protocol is designed in a Zero Trust architecture that utilizes blockchain to decentralize trust. However, since it does not rely on trusted hardware components to assure strong integrity protection, it remains vulnerable to remote spoofing attackers originating from the network layer. In contrast, hardware-assisted solutions limit the exploitability only to adversaries with physical attack vectors; only attackers capable of physically tampering a target device might actually compromise it.

In [22], a PUF-based authentication and data integrity protocol is proposed, which leverages blockchain for storing challenge-response pairs (CRPs), enabling continuous authentication of field sensors in SCADA. The protocol is restricted to SCADA systems operated by a single authority and, therefore, is not suitable for multiauthority environments. In [23], a pairwise continuous authentication protocol is proposed which refreshes the CRPs in every protocol round. In [24], the proposed PUFDC protocol uses a static authentication mechanism and then based on device location performs a continuous authentication. In [25], a blockchain-based authentication for industrial IoT (IIoT) is proposed that uses PUF capabilities for identifying the devices. A centralized certificate authority is used for issuing device certificates, which strongly limits decentralization and maintains strong trust assumptions. In [7], a fog nodes layer is proposed to address the resource limitation of IoT devices in processing heavy computational tasks for a group of IoT devices. The protocol utilizes an encrypted channel established through mutual initial authentication between IoT and fog nodes and integrates blockchain with a Proof of Authority consensus algorithm for the continuous authentication phase. Qian et al. [26] presented a hybrid PUF-based consortium blockchain for IIoT authentication. The proposed system supports different devices to generate different types of PUFs which synthesize the consortium blockchain. It also supports multiple endorsers, which could potentially include multiple authorities. Unfortunately, it does not support dynamic connectivity changes and is not suitable for continuous authentication. The authentication protocol of [27] uses device IDs, timestamps, and CRPs generated by the PUFs to authenticate sensor nodes to medical providers. The protocol, however, does not

TABLE I
COMPARISON OF STATE-OF-THE-ART CONTINUOUS IoT
AUTHENTICATION PROTOCOLS

Papers	Multiple authorities	Dynamic connectivity	Gateways' trust	Attack vector	Sybil attack protection
[2]	✗	✓	By assumption	Network	✗
[5]	✗	✓	By assumption	Network	✓
[7]	✗	✓	N/A	Network	✗
[20]	✓	✓	By assumption	Network	✗
[21]	✗	✓	N/A	Network	✗
[22]	✗	✓	N/A	Physical	✓
[23]	✗	✓	By assumption	Physical	✓
[24]	✗	✓	Zero-Trust	Physical	✓
[25]	✗	✓	By assumption	Network	✗
[26]	✓	✗	N/A	Physical	✗
[27]	✗	✓	By assumption	Physical	✗
HA-CAAP	✓	✓	GW attestation	Physical	✓

protect against Sybil attacks. Furthermore, it is designed for single-authority environments and, therefore, is not suitable for multiparty ecosystems. In summary, as shown in Table I, most existing continuous IoT authentication protocols are designed for centralized environments where all devices and gateways are operated by a single authority ([2], [5], [7], [21], [22], [24], [25], [27]).

Recent studies have explored the use of attestation mechanisms based on TEE. Devices equipped with hardware TEE modules are capable of enforcing strong trust (e.g. [28], [29], [30], and [31]). In other works, blockchain technology has been employed to ensure the immutability of attestation mechanisms, e.g., [32], [33], and [34].

The works of [20] and [26] are conceptually closest to this work. They both integrate software and hardware components to support authentication in multiauthority settings. In [26], a platform that utilizes a consortium blockchain and a hybrid PUF is proposed, to authenticate devices in industrial systems. In [20], a context-aware continuous authentication method is proposed, whose goal is to authenticate users to IoT domains, based on the context information of smartphone sensors. While these systems offer secure, hardware-supported continuous authentication, they rely on strong trust assumptions for the gateways. In addition, they do not support a mechanism to detect Sybil attacks. Our protocol addresses these gaps by providing robust trust assurances for the gateways executing the continuous authentication protocol, via a continuous gateway attestation mechanism. In addition, the blockchain network operated by the gateways, provides a real-time device inventory, to prevent Sybil attacks at different network domains.

III. HA-CAAP PROTOCOL

Assumptions and High-Level Description: We consider a multidomain network comprising multiple gateways and IoT devices belonging to different authorities, with dynamic connectivity changes (see motivating scenarios in Section I). Concerning the involved hardware, we make assumptions that are typical for mainstream gateways and IoT sensors. We assume that: 1) gateways have both trusted and untrusted execution modules. Although they are equipped with TEE modules that support hardware RoT attestation to secure access to crypto keys, due to hardware and/or functional limitations, most of its software is executed in the untrusted (typical OS) execution environment and 2) due to their

inherent limitations, the IoT devices are not equipped with a TEE; however, we assume they are equipped with a PUF mechanism that offers resistance to cloning attacks. The PUF, however, is not considered as a TEE since PUFs do not provide full immunity to attacks. For example, modeling, side-channel and physical tampering attacks are typically assumed as feasible attacks for PUF-based devices.

The main goals of this protocol are to: 1) *continuously authenticate devices* that may be connected to different gateways at different times; 2) allow *secure communication* between the gateways and their dynamically changing group of connected devices; 3) *continuously attestate gateways* to support their trusted execution; and 4) allow *real-time inventorying* of IoT devices by all gateways, in order to minimize the impact of a corrupted node on a single instance and prevent Sybil attacks of cloned devices. An overview of the protocol is depicted in Fig. 1, while Table II summarizes the notation.

Initially, all trusted gateways form a private blockchain network. The private blockchain enables gateways to securely share and maintain lists of known gateways and devices. The *GatewaysList* (*GWL*) stores information about the hardware RoT certificates of trusted gateways. The *EnrolledDevicesList* (*EDL*) stores the details of all enrolled IoT devices, such as their unique identifiers and fingerprints. Additional devices can be added in the *EDL* only through an already trusted gateway. In the case of IoT devices, their fingerprints are extracted and added in the *EDL*, and in the case of gateways, their hardware RoT public keys are added in the *GWL* (details are given in Section III-A). Finally, the *ActiveDevicesList* (*ADL*), includes those devices that are currently active (i.e., connected to a gateway). The gateways continuously update this list in the blockchain, to record which devices are currently connected to them.

Each gateway periodically broadcasts a fresh challenge to its nearby devices (see Section III-B). All devices receiving the challenge must successfully respond, by using their PUF (and other optional secret information shared with the gateway). Based on the responses, each gateway continuously (re)authenticates its connected devices, generates a group key and shares it with all authenticated devices. For each gateway, an *epoch* is defined as the time period during which the group of devices connected to the gateway is stable. If, during a periodic authentication request, the gateway detects a change in its connected devices' group (joins or leaves), this event will trigger an epoch change for the specific gateway. As a result, the gateway forms a new group key, based on the responses of all authenticated devices, and shares it within the new group.

In addition, the gateway publishes a new transaction on the blockchain, to update the *ADL* with its group of connected devices for the new epoch. By continuously checking the *ADL* before adding a node to their updated group, gateways have access to real-time device inventories for all other gateways (possibly managed by different authorities) and prevent Sybil attacks (details are given in Section III-D). Finally, trust between the gateways is supported by a continuous gateway attestation mechanism (see Section III-C).

Threat Model: As already noted in the assumptions, gateways are assumed to be equipped with a TEE, which protects

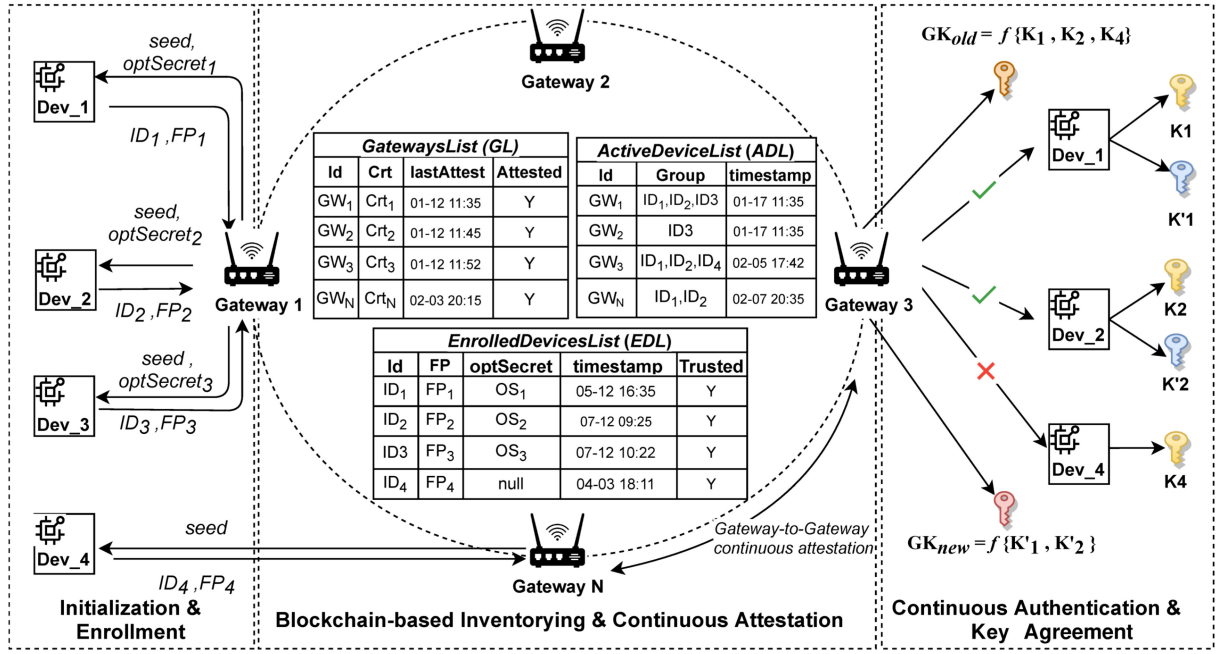


Fig. 1. HA-CAAP: high-level architecture.

keying material and attestation, while the rest of their functionality is executed in a typical execution environment. On the other side, the IoT devices may become corrupted. Adversaries with physical access to an IoT device are assumed capable of fully corrupting the device (e.g., extract PUF and all secret information). However, remote adversaries cannot tamper the devices' PUF. Our goal is to secure the network from the following attacks.

- 1) *Spoofing Attack Protection*: The protocol must prevent unauthorized access to IoT devices. Only legitimate enrolled devices must be able to authenticate to gateways. Attacks outside the protocol itself, such as software and hardware vulnerabilities that may lead to node corruption, are considered out of scope.
- 2) *Cloning Attack Protection*: Adversaries with physical access that have successfully corrupted a device, should not be able to clone the corrupted node and add clones into the network without being detected.
- 3) *Sybil Attack Protection*: An extension to the above is Sybil attack protection. A corrupted node should not be able to present multiple identities and connect simultaneously in multiple gateways.

Thus, our main goal is to minimize the impact of a corrupted node on a single instance and prevent adversaries from cascading node corruption.

A. Initialization and Enrolment Phase

During the initialization phase, we assume that each participating authority (i.e., owners of gateways and IoT devices) agreeing to run the protocol, possesses an authority public key certificate. Gateways belonging to trusted authorities form a private blockchain network. Each gateway in the network serves as a blockchain node and connects to others in a

peer-to-peer manner. The blockchain enables the gateways to share and dynamically update the *EDL*, the *GL* and the *ADL* defined above (see Table II). In addition, the public key certificates of all the authorities are added in the blockchain, to allow authorities to cross-verify their validity.

Gateways are enrolled by trusted administrators having access to the authority's private key as follows: for each gateway, the relevant authority chooses a unique identifier GW_X and a unique RoT private/public key pair gsk_X, gpk_X , generated within the gateway's TEE. The private key always rests within the TEE module. The public key is signed with the authority's private key, to generate a certificate $certGW_X$. For each enrolled gateway, an entry $[GW_X, certGW_X, timestamp]$ is added to the *GL*.

A device can be added to the enrolled devices' list *EDL* by a trusted gateway. The goal is to bootstrap a trust relation between a gateway and a device, by securely sharing some secret information that can be used for device authentication. The procedure is carried out as follows: Let D_i denotes a device uniquely identified by ID_i , and let GW_X denote a trusted gateway X . Initially, each device is prompted by a trusted gateway in a secure environment. A random value, denoted as *seed*, is generated from the gateway and given to the device to set the initial state of the PUF circuitry, ensuring the unpredictability and uniqueness of the output, as shown in Fig. 2. In addition, if the underlying PUF mechanism is considered weak for authentication purposes, an optional secret, *optSecret* is chosen by the gateway and is securely stored on the device side. The goal of *optSecret* is to add sufficient entropy to the exchanged secret information and to allow for exponentially many CRPs, if this is not supported by the underlying PUF mechanism. Otherwise, it may be neglected (or it may be used as a second authentication factor).

TABLE II
PROTOCOL NOTATION AND MAIN FUNCTIONS

Notation	
GW_X	Gateway X .
D_i	Device with ID_i .
FP_i	The unique fingerprint based on the stable registries for each device D_i extracted during enrollment phase.
$optSecret_i$	A pairwise secret, optionally shared between D_i and the gateway during the enrollment phase.
$GatewaysList(GWL)$	List of all enrolled gateways and the RoT certificates.
$EnrolledDevicesList(EDL)$	List of all enrolled devices along with their fingerprints.
T	A pre-defined period for periodic continuous authentication.
e	Epoch counter; updated independently for each gateway, when a modification in the group of its connected devices is detected. An epoch may include one or more periods T .
$t[e]$	The start time of epoch e .
t_{cur}	Temporal variable, used to check for epoch changes.
$Group_X[e]$	A list containing all devices connected to gateway X at the epoch e .
$K_i[e]$	A pairwise key shared between D_i and gateway X for the epoch e .
$GK_X[e]$	The group key shared between GW_X and all devices in $Group_X[e]$ for the epoch e .
$ActiveDeviceList (ADL)$	Dynamic list containing the history of the connection state of all devices, grouped per gateway. A gateway GW_X will update its device list in ADL (by calling the <i>publishInADL</i> function), if periodic authentication indicates an epoch change for GW_X .
gsk_X, gpk_X	Gateway's X private/public root-of-trust (RoT) key pair.
Main functions	
$Enroll(seed)$	Outputs the device's unique fingerprint FP_i for the given <i>seed</i> .
$GenPUF(chal)$	Outputs a unique value $FP_i[chal]$, based on the device's fingerprint and a challenge.
$GenDevKey(FP_i[chal], optSecret_i, nonce)$	On input $FP_i[chal]$, a random <i>nonce</i> and optionally a shared secret <i>optSecret</i> , it outputs a key K_i . Instantiated by a secure key derivation hash function.
$GenResponse(K_i, nonce)$	On input a key K_i and a nonce, it outputs a string $resp_i$ linked to the key K_i . Instantiated by a secure one-way hash function.
$searchInADL(ID_i)$	Executed by a gateway, to search in ADL for the current connectivity status of a device requesting to connect.
$raiseAlarm(ID_i)$	Executed by a gateway, to publish an alarm in the blockchain, if authentication for the device with ID_i fails.
$publishInADL(X, e, Group_X[e], t[e])$	Executed by gateway GW_X to publish in ADL an epoch change. It triggers the devices' watchdog to search for Sybil nodes in all gateways lists.

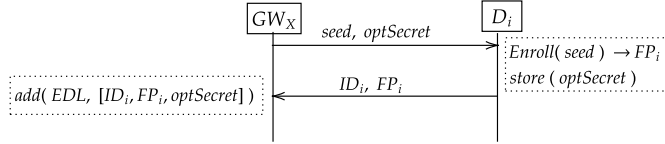


Fig. 2. Device enrollment phase.

Then, a fingerprint is extracted from the device that acts as a generator of responses to each given input. The extraction is accomplished with the use of $Enroll(seed)$ function. This function accepts a random value as input and generates a binary sequence consisting of 0s and 1s, which is derived from the stable registries of the PUF. For each successfully enrolled device, the enroller gateway will add a unique entry in the EDL , of the form $[ID_i, FP_i, optSecret_i]$. This is implemented by storing the relevant transaction in the current block of the private blockchain. To link all relevant transactions (EDL entries), a pointer is added in each new EDL entry, pointing to the block containing the previous entry.

Remark 1 (Revocation of Devices): If needed, gateways may also add revocation entries for already enrolled devices similarly. A newer entry in the EDL will supersede any previous entry for the same device.

Remark 2 (Fine-Grained Access to Device Secret Information): It is easy to limit access to device secret information (FP , $optSecret$), for example to particular gateways or authorities, by applying group or attribute-based encryption before publishing entries in the EDL . The relevant decryption keys will then be stored in the TEE of those gateways that have been granted access to the particular IoT devices.

B. Continuous IoT Authentication and Key Agreement

During this phase, a legitimate gateway having access to the list of all valid devices (enrolled in the EDL), periodically challenges all nearby active IoT devices, to authenticate them based on their PUF. In addition, the gateway will securely establish pairwise keys and a group communication key with all successfully authenticated devices. When the gateway detects a change in its device group membership during a periodic authentication cycle, it will update the relevant pairwise keys and the group key to a new one and share it with its updated group of connected devices. The protocol is described in Fig. 3.

1) Initialization: Let T be a predefined time period that defines the period for continuous authentication. Depending on the application needs and the capabilities of the devices, T may vary from a few seconds to minutes. For each gateway, an *epoch* is defined as the time period during which the set of devices connected to the gateway is stable. Let e denotes the epoch counter for a gateway X and let $Group_X[e]$ denote the list of all devices connected to gateway X at epoch e . At each period T the gateway executes a new authentication cycle and if device connectivity modification is detected, then this triggers an epoch change for the particular gateway. Epochs may change independently for each gateway based on their dynamic connectivity changes. On startup, the gateway initializes its epoch counter e to 0, and the start time of epoch 0 with the current clock time. The temporal variable t_{cur} is also initialized to the current clock time. Then, the gateway continuously sends authentication requests to all its nearby devices every period T , to detect membership changes.

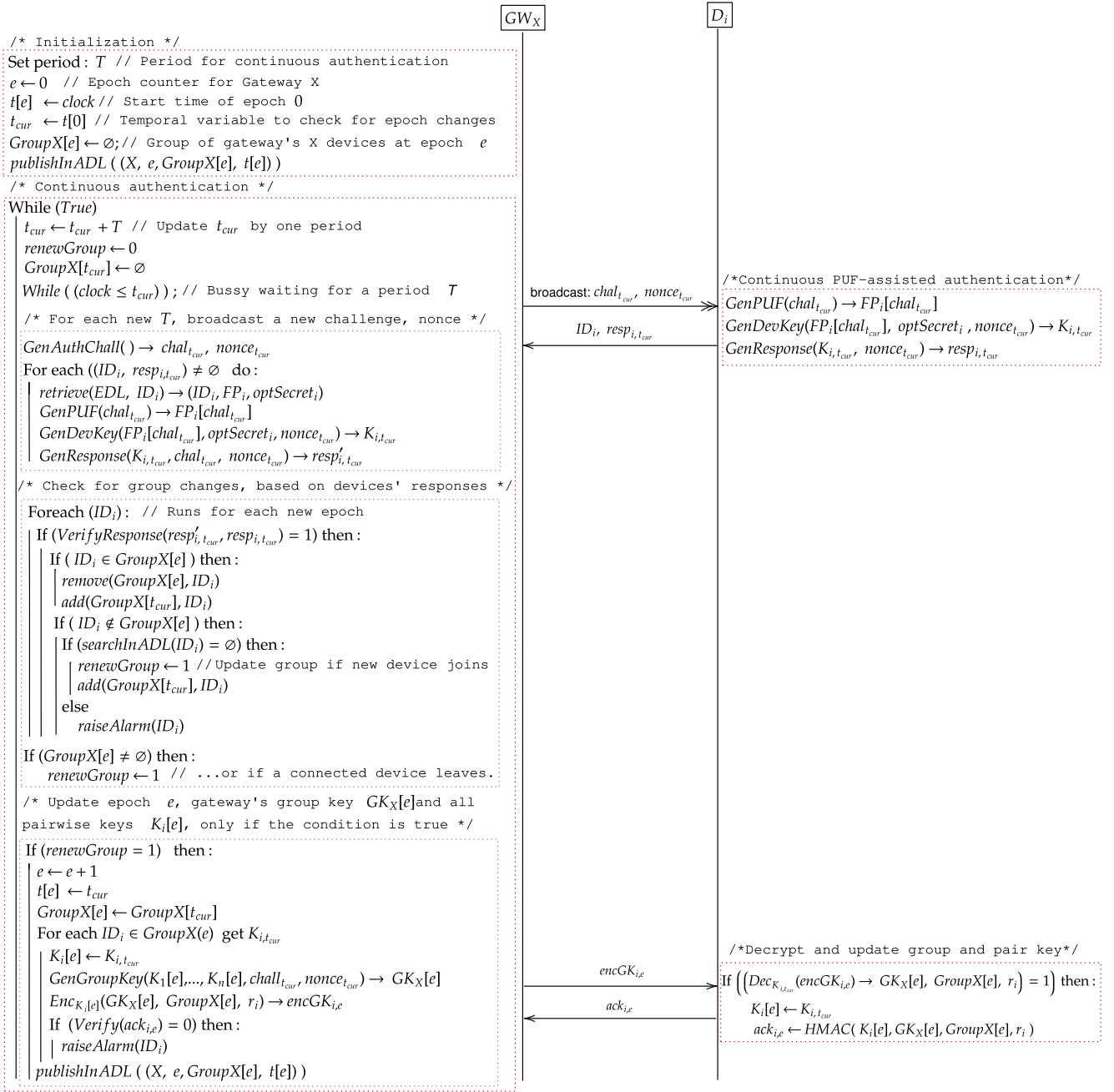


Fig. 3. Continuous authentication and group key agreement protocol.

2) *Continuous Authentication (Gateway Side)*: At the beginning of each authentication cycle, the gateway increases t_{cur} by T , sets a boolean variable $renewGroup$ to 0, and a temporal empty list $GroupX[t_{cur}]$ is created. This list will store the IDs of all devices connected to the gateway at the time t_{cur} under examination. If any change is detected between $GroupX[t_{cur}]$ and the group of devices for the current epoch, then the boolean $renewGroup$ will become 1, the epoch counter e will be increased by 1, and $GroupX[t_{cur}]$ will become $GroupX[e]$ for the new epoch. The gateway waits for time T and then broadcasts a random $\{chal_{t_{cur}}, nonce_{t_{cur}}\}$, which triggers an authentication cycle.

3) *Continuous PUF-Assisted Authentication (IoT Side)*: On the devices' side, any device D_i that is in range and is already connected to X or wishes to connect to the gateway, must (re)execute PUF-assisted authentication. The device will first run $GenPUF(chal_{t_{cur}})$, to output its corresponding part of the fingerprint $FP_i[chal_{t_{cur}}]$. Then, the device will execute the $GenDevKey(FP_i[chal_{t_{cur}}], optSecret_i, nonce_{t_{cur}})$ to produce a unique secret key $K_{i,t_{cur}}$ based on the fingerprint, which will be shared between the device and the gateway. If exists, $optSecret$ is used to increase the entropy for key generation. Then, the device runs $GenResponse(K_{i,t_{cur}}, nonce_{t_{cur}})$ to generate the response $resp_{i,t_{cur}}$ and send it to the gateway. On the gateway

side, a similar procedure is followed for each response, retrieving from the *EDL* the fingerprint FP_i (and $optSecret_i$, if used), recomputing and verifying the expected response.

4) *Detecting Group Changes*: Then, the gateway will check all the verified responses, to see if there is a connectivity change. A change may occur either if a device that was previously in the group has not responded (leave of a device); or if a device that did not exist in its current group $GroupX[e]$ has been successfully authenticated (new join). For the latter case, the gateway additionally checks in the blockchain to see if the new device concurrently belongs to the group of another gateway, with the use of the *searchInADL()* function (more details are given in Section III-D). This check will prevent Sybil attacks by compromised devices. In either of the two cases, *renewGroup* will be set to 1 to indicate an epoch change. Otherwise, the epoch does not change.

5) *Epoch and Key Update*: If a modification in the gateway's group is detected, the epoch counter e will be incremented. In that case, the gateway will update the pairwise keys for all successfully authenticated devices, by setting the temporal pairwise keys as the pairwise devices' keys for the new epoch ($K_i[e] \leftarrow K_{i,t_{cur}}$). Also, it will update the group key $GK_X[e]$ for the new epoch, via the *GenGroupKey()* function. The function takes as input all the updated pairwise keys, $K_i[e]$ and the $chal_{t_{cur}}, nonce_{t_{cur}}$ pair. Subsequently, the updated group key $GK_X[e]$ is encrypted with the updated pairwise keys $K_i[e]$, and is transmitted to each device belonging to the group. In order to verify that each device D_i has successfully received key, a random nonce r_i is used for each pairwise encryption. Finally, the gateway calls the *publishInADL* function to publish in the blockchain an entry for its new epoch, which includes the new epoch counter for GW_X , the new list of the connected devices $GroupX[e]$ and the start time of the new epoch $t[e]$. Note that the continuous authentication cycle runs each period T , while epoch changes (and the pairwise and group key update) will only happen in case a change in the group membership is detected.

Remark: Devices leaving a group can be programmed to notify the gateway and to trigger protocol runs before the expiration of the period T .

C. Continuous Gateway Attestation

Assuring the trustworthiness of the gateways is crucial in multiauthority environments, since a compromised gateway would be able to enroll fake IoT devices and add them in the *EDL*. Attestation provides a powerful mechanism for ensuring the security and trustworthiness of a hardware platform, enabling remote entities to verify the identity and integrity of a platform before establishing a connection or allowing access to sensitive resources. HA-CAAP prevents compromised gateways from joining the system, through a continuous gateway attestation protocol distributedly run among the gateways.

Fig. 4 presents the proposed decentralized continuous attestation protocol. Let T_{att} denotes the attestation period. Each gateway executes the attestation protocol within its protected TEE, which enforces it to request attestation within each period T_{att} . Recall from Section III-A that for each trusted

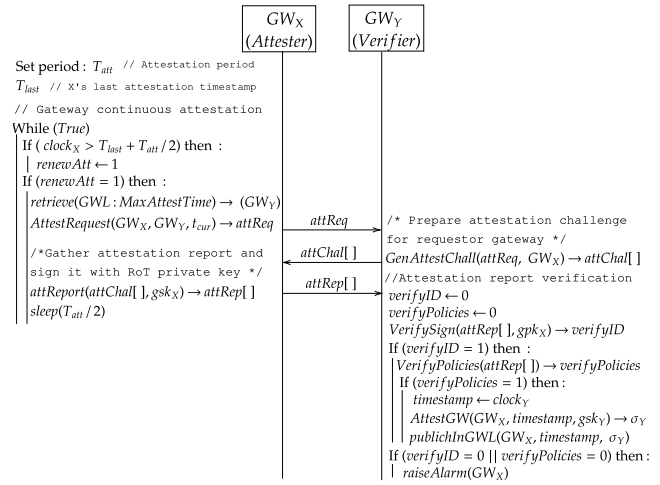


Fig. 4. Continuous gateway attestation protocol.

gateway, an entry is maintained in the *GatewaysList* (*GWL*), containing its unique id GW_X , its RoT public key gpk_X and its latest attestation timestamp (see also Fig. 1). In our protocol, for each attestation request, the role of the verifier is distributed among the gateways that have been attested for the current period. Let GW_X be a gateway reaching near reattestation time (i.e., $renewAtt = 1$). The attester GW_X will access the *GWL* in the blockchain, and based on the attestation timestamps, the gateway with the maximum remaining attestation time will be chosen as the verifier. Let GW_Y be the gateway identified as the verifier for the specific protocol run. Then, GW_X sends an attestation request to GW_Y . The verifier prepares an attestation challenge $attChal$ and sends it to the attester gateway, including a unique challenge value and various system state attributes.

The attester GW_X prepares the attestation report. The claims of the attestation report consist of the hardware, firmware and software versions and their hash values collected by secure booting together with the challenge value. These claims are then digitally signed using gateway's GW_X private RoT key gsk_X , which resides within its immutable read-only and secure memory (TEE). The signed claims form the attestation report $attRep$ of GW_X , and are transmitted back to the verifier gateway GW_Y . Upon arrival of the attestation report, GW_Y will verify the identity of the attested gateway using its public key stored in the *GWL*. At this stage, the verifier may force policy requirements on the attested gateways, such as minimum hardware, firmware, and software versions. If the attested gateway successfully passes both checks of identity and policies, the verifier GW_Y signs an updated attestation entry with an updated attestation timestamp, and publishes it in the *GWL*.

D. Real-Time Device Inventorying and Gateway Monitoring Through Blockchain Protocol Integration

At the gateways' side, the hardware immutability of the TEE assures that RoT keys and enclaved protocol functions remain protected.

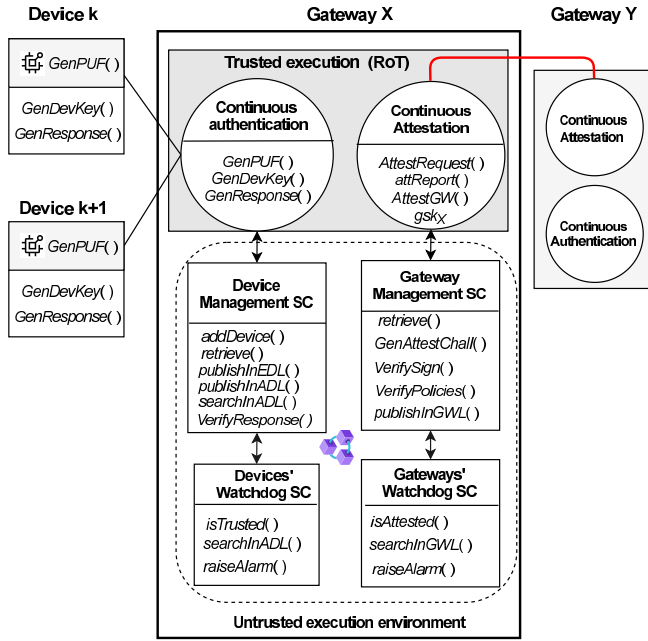


Fig. 5. Blockchain-based gateway monitoring and device inventorying: blockchain immutability assures execution integrity and synchronization for those parts of the protocol that are executed outside the TEE.

However, since HA-CAAP assumes distributed execution resulting in dynamic changes of shared information, various protocol functions cannot be executed within the isolated TEE component. For this reason, blockchain technology is used as an *integral component* of HA-CAAP design; the immutability of the blockchain will assure execution integrity and synchronization, for those parts of the protocol that cannot be hardware-protected.

The use of a private blockchain allows for: 1) key management interoperability between different authorities; 2) real-time device inventorying, to prevent Sybil attacks; and 3) real-time monitoring of gateways to prevent attestation violations. In Section III-A we have already described how blockchain supports the interoperability of authority certificates and gateway public keys, during the initialization. Below, we will describe the functionalities supported by blockchain integration.

1) *Real-Time Device Inventorying*: As shown in Fig. 5, for the continuous authentication protocol described in Section III-B, the functions requiring access to secret keying material (*GenPUF*, *GenDevKey*, *GenResponse*) are executed within the TEE. This, however, is not possible for those functions that require distributed read/write access to shared volatile information, such as the *ADL*, *EDL*, and *GWL* lists.

Since gateways are nodes of a private blockchain, these functions are implemented by smart contracts supporting protocol implementation. The device management smart contract (DMSC) implements all the protocol functions that update or search within the *EDL* (*publishInEDL()*, *retrieve()*) and the *ADL* (*publishInADL()*, *searchInADL()*).

The blockchain immutability and consensus mechanisms, ensure that gateways will update the lists as expected by the protocol.

However, since gateways concurrently execute the protocol in different execution environments, race conditions may occur; it is possible that gateways read information from the *ADL*, before other gateways have updated their own active devices list. A corrupted device could exploit gateway synchronization to concurrently connect in multiple gateways, without getting noticed in real-time. A cloned or malicious device concurrently sends join requests to two or more gateways. For example, GW_X and GW_Y run *searchInADL()* almost concurrently and then they both update their list (*publishInADL()*) without realizing that the same node joined in both. The Sybil node will not be detected in real-time. Due to the distributed execution, typical process synchronization solutions (e.g., semaphores or memory locks) cannot be applied.

To prevent this, we exploit the smart contract triggering mechanism. The devices' watchdog smart contract (DWSC) is implemented and it is triggered by the DMSC, after every call in *publishInADL()* is made. For each modification in *ADL*, the DWSC will run the function *isTrusted()*, which checks for all devices contained in a modified active devices list of a gateway, if a Sybil node exists in any other gateways' list. In this way, it is assured that the DWSC will detect a Sybil attack in real-time.

2) *Real-Time Gateway Monitoring*: Similarly, for the continuous attestation protocol described in Section III-C, the RoT keys and functions requiring access to them are executed in the TEE. The TEE part of the protocol assures that an attestation request is executed within the attestation period T_{att} .

The functions that either require read/write access to the *GWL* (*retrieve()* and *publishInGWL()*) or only require verification based on public keys *VerifySign()*, *VerifyPolicies()* are executed in the blockchain, by the gateway management smart contract (GMSC) (see Fig. 5). The GMSC ensures that the most recently attested gateway will be selected for the signature verification procedure and policy enforcement.

As in the previous case, the smart contract triggering mechanism is utilized to monitor attestation in real time. A similar watchdog is triggered after each attestation request, in order to verify that all gateways have been successfully attested within each attestation period T_{att} . When a gateway (say GW_X) requests attestation from another gateway GW_Y the gateways' watchdog smart contract (GWSC) is triggered to check the *GWL* for gateways that are not attested or have reached the attestation time limit. In this way, gateways with expired attestation (e.g., that were previously offline) are forced to attest before joining again.

IV. SECURITY ANALYSIS

Assumptions and Adversarial Model: We assume a probabilistic polynomial-time (PPT) adversary $\mathcal{A}$, who is able to eavesdrop and tamper the communications between devices and gateways. The adversary has access to polynomially many protocol runs $\{chal, nonce, resp, encGK, ack\}$. For spoofing attacks, we assume that the adversary does not have physical access to the device. For cloning and Sybil attacks we relax

this assumption and the adversary is allowed to physically tamper a device. The information extracted by the compromised device can then be used to assist the adversary in performing cloning and Sybil attacks. We assume that the underlying crypto primitives (Enc , $HMAC$) are secure. Finally, we assume that the underlying blockchain has high immutability; updates in the blockchain are possible only through the consensus mechanism given that the majority of the nodes adhere to the blockchain execution.

High-Level Description: Our security analysis strategy is as follows. First, we will prove that the PUF continuous authentication protocol is secure against spoofing attacks, i.e., only legitimate nodes that have already executed the enrollment phase are able to correctly execute the PUF-based authentication phase, provided that the underlying PUF and the underlying crypto blocks are secure. Then, we will focus on the analysis of the security requirements of the PUF component. We will show that it suffices to use, either a strong PUF, or a weak PUF in combination with a preshared secret of sufficient length. Then, we will extend our analysis to show how the blockchain-based inventorying mechanism can protect from Sybil attacks. Finally, we will examine the security of the continuous gateway attestation mechanism.

A. Spoofing Attack Protection

Authentication Bypass Security Experiment: Let Π denotes the proposed authentication protocol. The security analysis is based on a security game $AuthExp_{\mathcal{A}, \Pi}(Devices[], chal, nonce)$ between a Challenger and the Adversary, that simulates an attacker aiming to break the authentication phase of the protocol. The experiment comprises the following phases.

Initialization Phase: The challenger runs the preauthentication phase $\forall D \in Devices[]$, to enroll all legitimate devices. The identifiers of all legitimate devices are given to the adversary.

Build Phase: $\mathcal{A}$ chooses a target device ID_v and sends it to the Challenger.

Challenge Phase: The Challenger chooses a challenge $chal, nonce$, and runs the authentication phase with polynomially many legitimate devices (but not the target device), until all responses $resp_i$, ($i \neq v$) are received. Then, the Challenger runs the verification phase of the protocol for all devices (including the target device ID_v) and generates the corresponding group key GK . Then, the Challenger creates all the pairwise encryptions of the group key for all the participants (including the target device). Finally, the Challenger gives to $\mathcal{A}$ the challenge tuple $chal, nonce$ and the encrypted group key $enkGK$ (encrypted with the presumed key K_v for the target user).

Guessing Phase: The adversary gives the input to an oracle $\mathcal{O}$ and the oracle outputs a tuple $(\tilde{resp}, \tilde{ack})$. The adversary outputs a bit $b \in \{0, 1\}$, where: $b = 1$ if $(\tilde{resp}, \tilde{ack})$ correspond to the output $(resp_v, ack_v)$ of the execution of Π for the given challenge $chal, nonce$ and for the target device ID_v ; otherwise $b = 0$.

Definition 1: The advantage of an adversary $\mathcal{A}$ to break the authentication protocol Π is defined as

$$\begin{aligned} Adv_{\mathcal{A}, \Pi} &= Pr(AuthExp_{\mathcal{A}, \Pi} \rightarrow 1) \\ &= Pr((\tilde{resp}, \tilde{ack}) = (resp_v, ack_v)) = \epsilon. \end{aligned} \quad (1)$$

Theorem 1 (Spoofing Attack Protection): Assuming that $GenPUF$ is a secure PUF function with an output of sufficient entropy, $GenDevKey$ is instantiated by a secure key derivation hash function, $GenResponse$ is instantiated by a secure one-way hash function and the underlying cryptographic building blocks $HMAC$ and Enc are secure, then $\mathcal{A}$ can succeed in breaking Π only with a negligible advantage.

Proof: Suppose that $\mathcal{A}$ succeeds in breaking $AuthExp$ with a nonnegligible advantage ϵ . We will show that $\mathcal{A}$ can be used as a subroutine by a PPT algorithm $\mathcal{B}$ to break the underlying cryptographic primitives Enc and $HMAC$ with the same probability.

The adversary runs the experiment $AuthExp$. As described above, $\mathcal{A}$ chooses a target user ID_v at the build phase. During the challenge phase, $\mathcal{A}$ is given the challenge $chal, nonce$. During the guessing phase, $\mathcal{A}$ queries the oracle $\mathcal{O}$ with the challenge and it receives $(\tilde{resp}, \tilde{ack}) = (resp_v, ack_v)$ and outputs $b = 1$ with nonnegligible advantage.

We distinguish two cases. In the first case, the Adversary is able to generate the key K_v corresponding to the target device and the given challenge. In the second case, the Adversary wins the security game, without generating the key K_v first.

Case 1: Clearly, if $\mathcal{A}$ can construct the correct key K_v of the target device for the given challenge, then it is trivial to construct the correct response and group key acknowledgment. However, note that the key K_v is generated as the output of a secure key derivation function $GenDevKey$, taking as input the output of $GenPUF$ (and optionally the secret $optSecret$), which is assumed to be a secure PUF of sufficient entropy. Thus, $\mathcal{A}$ cannot directly construct K_v without contradicting to the security assumptions of the PUF. In addition, even if $\mathcal{A}$ has eavesdropped a response $resp_v$ generated by the legitimate device, the Adversary will not be able to invert $resp_v$ and learn K_v , without contradicting the security assumption for $GenResponse$ (a secure one-way hash function).

Case 2: Assume that $\mathcal{A}$ outputs the correct values $resp_v, ack_v$, without first computing the key K_v . In that case, $\mathcal{A}$ can be used as a subroutine by an algorithm $\mathcal{B}$ to break the underlying cryptographic primitives Enc and $HMAC$ with the same advantage (see Fig. 6). The goal of $\mathcal{B}$ is, given a challenge ciphertext $c = Enc_K(m)$ for unknown K and m , to output the message authentication code $HMAC(K, m)$, which implies breaking the underlying primitives.

The Challenger sets $m = (GK, curGroup, r_v)$. Then, it encrypts m with K_v as $c = Enc_{K_v}(m)$ and gives the challenges c (for $\mathcal{B}$) and $chal, nonce$ (for $\mathcal{A}$). Now $\mathcal{B}$ forwards to $\mathcal{A}$ the challenge $chal, nonce$, who successfully solves it with a nonnegligible advantage ϵ , by running the security game $AuthExp$, and returns $resp_v, ack_v$. Now $\mathcal{B}$ forwards to the Challenger ack_v (which is equal to $HMAC_{K_v}(m)$) and successfully responds to the challenge c , with the same nonnegligible advantage ϵ . ■

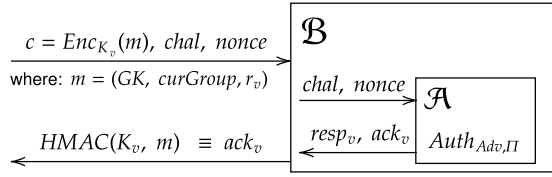


Fig. 6. Reducing spoofing attacks to attacks against the underlying crypto primitives *HMAC* and *Enc*.

In the analysis above, we considered that the underlying PUF is strong with sufficient entropy. We relax this assumption by allowing additional sources of randomness to be combined along with the PUF output. The constraint here is that the combination of the PUF output and the shared secret should have sufficient entropy. In practice, this implies that the proposed protocol could be implemented using a weak PUF, provided that the device and the gateway also share additional secret information.

Theorem 2 (Resistance to PUF-Specific Attacks): The underlying PUF utilized in Π is sufficiently secure if: 1) either *GenPUF* is a strong PUF function having an output of sufficient length and entropy or 2) the concatenation of the output of *GenPUF* and *optSecret* is a bitstring of sufficient length and entropy.

Proof: We examine both cases.

Case 1: This comes directly from the proof of Theorem 1 (case 1). If *GenPUF* is a strong PUF of sufficient entropy, then it is secure from brute-force and modeling attacks.

Case 2: In that case, *GenPUF* is instantiated by a weak PUF, which may be vulnerable to brute-force and modeling attacks. We examine these attacks.

Resistance to Brute-Force Attacks: Assume that the input space of *GenPUF* is tractable in polynomial time. In that case, $\mathcal{A}$ is able to compute all possible *RP* values for a given challenge, by running *ReorderPUF* polynomially many times. However, by adding a shared secret *optSecret* such that $(FP[chal], optSecret)$ is a bitstring of sufficient length and entropy (e.g., ≥ 128 bit), then $\mathcal{A}$ will not be able to exhaustively compute all possible keys K through *GenDevKey*, in polynomial time.

Resistance to Modeling Attacks: Assume that *GenPUF* is weak and vulnerable to modeling attacks. To train an attack model, the attacker requires access to sufficiently many challenge/response pairs. However, the response *resp* captured by a modeling attacker, is the output of *GenResponse*, which is a secure one-way hash function with input the secret key K . In turn, K is generated by *GenDevKey* taking as input secret data $(FP[chal], optSecret)$ of sufficient entropy. Therefore, if *GenResponse* behaves as a random function and takes as input secret data of sufficient entropy, modeling attacks are not feasible. ■

B. Cloning and Sybil Attack Protection

As defined in our adversarial model above, although for spoofing attacks we assume that the adversary has network access to the device, for cloning and Sybil attacks we relax

this assumption and allow adversaries to also have physical access to the device. We formally analyze these attacks.

Theorem 3 (Cloning Attack Protection): An adversary who physically tampers a device and extracts the device's secret information $(FP, optSecret)$ will not be able to create new valid credentials $(\tilde{FP}, optSecret)$, provided that *GenDevKey* is instantiated by a secure one-way hash function.

Proof: Assume that $\mathcal{A}$ has access to the fingerprint and all secret information of a valid device (e.g., via physical tampering). Clearly, in that case, $\mathcal{A}$ would be able to extract $FP, optSecret$ from the corrupted node and create exact clones of the victim node. However, the adversary will not be able to use $FP, optSecret$ in order to construct clones with different credentials $\tilde{FP}, optSecret$ that result in the same key K of any other valid node. Finding different input values such that $GenDevKey(FP[chal], optSecret, nonce) = GenDevKey(\tilde{FP}[chal], optSecret, nonce)$ would imply that $\mathcal{A}$ is capable of breaking the second preimage resistance of *GenDevKey*, which is assumed to be a secure one-way hash function. ■

We proceed to analysis of Sybil attacks. Recall that the goal of a Sybil attacker is to present multiple identities and concurrently connect to multiple gateways, without getting noticed, using the same or different identities. Since a cloning adversary can only create exact clones of a compromised IoT device (with exactly the same secret information), below we examine Sybil attacks for that case.

Sybil Attack Security Experiment: Let Π_1, Π_2 , and Π_3 denotes the enrollment (Section III-A), continuous IoT authentication (Section III-B), and continuous Gateway attestation protocols (Section III-C), respectively. The security analysis of Sybil attacks is based on a security game $SybilExp_{\mathcal{A}, \Pi_2}(Devices[], chal, nonce)$ between a Challenger and the Adversary, that simulates a Sybil attacker, aiming to concurrently connect exact clones in different gateways, without getting noticed. The experiment comprises the following phases.

Initialization Phase: The challenger runs the preauthentication protocol $\Pi_1 \forall D \in Devices[]$, to enroll all legitimate devices. The identifiers of all legitimate devices are given to the adversary.

Build Phase: $\mathcal{A}$ chooses a target device ID_v and sends it to the Challenger. The Challenger gives to $\mathcal{A}$ all the secret information $FP_v, optSecret_v$ of the target device ID_v (the compromised device). The adversary uses the credentials to create an exact clone of the corrupted node, denoted as $\tilde{ID}_v$.

Challenge Phase: The Challenger concurrently initiates two instances of the authentication protocol Π_2 , one for Gateway GW_X and a second for another Gateway GW_Y .

Attack Phase: The adversary participates in both instances of the protocol, using the compromised node ID_v for GW_X and the exact clone $\tilde{ID}_v$ for the GW_Y protocol instance. If GW_X and GW_Y detect that $\tilde{ID}_v$ is a Sybil node of the corrupted node ID_v , then ID_v is revoked in the *EDL*. Otherwise, both instances are concurrently connected and the adversary wins.

We will show that $\mathcal{A}$ can win only with negligible probability, provided that the TEE of the gateways is secure and that the underlying blockchain utilized in Π_2 is immutable.

Definition 2: The advantage of an attacker $\mathcal{A}$ to break $SybilExp_A$ is defined as

$$Adv_{\mathcal{A}} = Pr(SybilExp_{\mathcal{A},D,\bar{D}} \rightarrow 1) = \epsilon. \quad (2)$$

Theorem 4 (Sybil Attack Protection): Assuming that the gateways' TEE is secure and that the underlying blockchain has a high level of immutability (i.e., can be updated only via the blockchain consensus mechanism), then Sybil attacks will be detected in at most T time, with overwhelming probability.

Proof: Recall from Section III-B that when a gateway publishes an update in its active devices list, the relevant *ADL* entry is signed with the gateways private RoT key within the protected TEE environment. In addition, as described in Section III-D the *ADL* is maintained in the blockchain and can be updated only via the Device Managements SC, (*publishInADL()*) also running in the blockchain. Finally, the Device Watchdog SC, is triggered after each update in the *ADL* to search for possible Sybil nodes for every device contained in the updated list. An adversary running *SybilExp* can win the security game with one of the following ways: 1) by managing to sign a fake *ADL* entry not containing the Sybil node (although the Sybil node is actually connected to the second gateway GW_Y); 2) by controlling the publishing of the *ADL* (i.e., compromising the *publishInADL()* function); and 3) by controlling the functions running in the Devices' Watchdog SC to avoid the triggering of the *isTrusted* function for all entries contained in the updated devices list of GW_Y . We examine all cases.

Case 1: Since the private RoT key of the gateway and the signing function of *ADL* entries are residing within the TEE, which is assumed to be trusted, then $\mathcal{A}$ can forge a signature only with a negligible advantage $\epsilon_1(N)$, where N is the key length of the RoT private key.

Cases 2 and 3: Each of these cases assumes that either DMSC or DWSC have been corrupted. However, since the underlying blockchain is assumed to be a secure blockchain with sufficient immutability, then $\mathcal{A}$ can modify the blockchain outside the consensus mechanism, only with a negligible advantage $\epsilon_2(I)$, where I is the immutability factor of the blockchain.

By combining all cases and Definition 2, it holds that $\epsilon \leq \epsilon_1(N) + \epsilon_2(I)$. Therefore, since the adversary can win only with negligible probability, Sybil attacks will be detected with overwhelming probability on each call of DWSC, which will be called for each update in *ADL* in every period T . ■

C. Gateway Attestation Protection

As described in Section III-C, gateway continuous attestation relies on the security of the TEE, which protects the use of the RoT private keys, and the Gateway Management and Gateway's Watchdog Smart Contracts, running within the secure blockchain. In a similar way as the one described in Section IV-B, it can be shown that assuming that the underlying TEE and blockchain are secure, then attestation bypass attacks will be detected in at most T_{att} time, with overwhelming probability.

V. EVALUATION

To evaluate HA-CAAP, we built a testbed consisting of a gateway, several IoT (sensor) devices, and a private blockchain. First, we describe the testbed implementation, focusing on the hardware RoT security primitives for both IoT devices (i.e., PUF) and gateways (i.e., TEE). Then, we evaluate the computation, network and energy efficiency of HA-CAAP, based on real-world requirements derived from a use case within the supply chain tracking market.

A. Testbed

A high-level overview of our HA-CAAP testbed is given in Fig. 7. Each gateway (Raspberry Pi) communicates via a proxy device (PyCom) with one or more sensor devices (PySense). The proxy device receives the sensor data over BLE and forwards them to the gateway over a serial connection. Each gateway connects to all other gateways using a GSM module. The gateway is equipped with a TEE. The TEE enables a hardware RoT that enclaves the crypto operations of the protocol. Outside the TEE, the gateway runs a node of the private blockchain, which supports the asynchronous protocol execution, monitoring and inventorying functionalities, via the smart contracts.

Gateway Devices: We implement the gateway functionality on a Raspberry Pi® 4 Model B platform (RPI), which contains a Broadcom BCM2711 chipset with 4GB of memory and a Quad Core 1.8GHz Cortex-A72 (ARMv8) 64-bit CPU. The RPi platform does not provide a hardware RoT, so we integrate the Infineon OPTIGA™ TPM 2.0 SLI 9670 evaluation board for the RPi as a trusted platform module (TPM). The TPM is responsible for secure key generation, storage and encryption. It ensures that each device has unique read-only authentication and attestation keys. The system runs Raspberry Pi OS, using Debian Bullseye 11, with secure booting implemented using U-Boot.¹ We use the TPM software stack (TSS)² and the TPM tools³ provided on GitHub.

Blockchain Implementation: To implement the private blockchain network among the gateways, we used the Hyperledger Fabric⁴ platform and developed all smart contracts shown in Fig. 5, in the Go programming language. The blockchain network was implemented using RPi devices as nodes. Each node was configured with a 64-bit Ubuntu Server 24.10 operating system, and Docker was utilized to construct and manage the Hyperledger Fabric environment. A total of 10 RPi devices were deployed: one device functioned as the Certificate Authority node, two devices were configured as peer nodes, two as orderer nodes, and the remaining five served as client nodes. Due to the ARM architecture of the Raspberry Pi, the Hyperledger Fabric binaries and Docker images were manually modified from source code.

In order to establish communication between the TPM and the blockchain, we used the Public Key Cryptographic Standard 11 (PKCS11) and the TSS implementation of the

¹<https://github.com/u-boot/u-boot>

²<https://github.com/tpm2-software/tpm2-tss>

³<https://github.com/tpm2-software/tpm2-tools>

⁴<https://www.hyperledger.org>

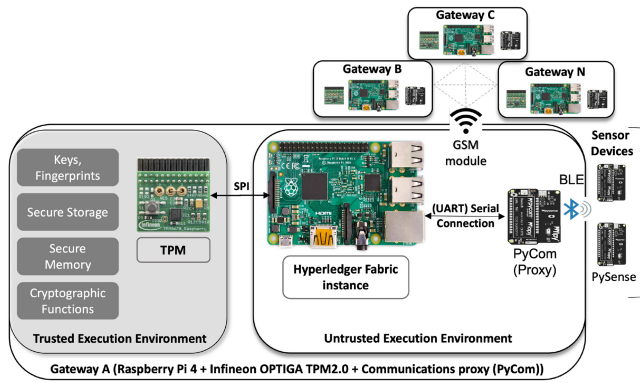


Fig. 7. Implementation architecture.

TCG group. The network connectivity modules are also running outside the TEE. In order to measure the performance of the blockchain, we employed the Hyperledger Caliper⁵ as our benchmarking tool. To simulate real-world conditions, we scaled up our system assuming the presence of 200 gateway devices, each of which runs a blockchain node. Based on the specifications provided, we determined that reading a transaction of 100 bytes of data through the blockchain takes approximately $t_r = 0.06$ s, while writing the same transaction requires approximately $t_w = 0.18$ s. The gateways communicate with each other on a network with an average speed of 2 Mb/s.

Gateway's Hardware RoT Mechanisms: Secure boot, encryption and continuous attestation are implemented using functions built on the TEE, separate from the sensor data handling and communications. In particular, we employ functions provided by the TSS which utilize the TPM in order to achieve security and integrity of information. Since the RPi bootloader is closed-source and to the best of our knowledge open source implementations are limited, we consider the RPi bootloader as trusted, and build our secure boot on top of it. Secure booting is implemented using the U-Boot bootloader, which is executed after the RPi bootloader but before the Linux kernel boots. The U-Boot bootloader utilizes the TPM for hashing the image that is about to boot and verifying that the image has not changed. The entire gateway image is hashed and signed using the hardware RoT keys that are only accessible within the TPM. This also assures hardware authenticity verification.

Secure attestation is performed periodically according to HA-CAAP. The attestation process (Fig. 4), is implemented similarly to the official Infineon Remote Attestation application note [35], as follows: GW_X provisions the public and private attestation keys (AK) using the TSS function `tpm2_createak`, specifying RSA, SHA256 and RSASSA-PSS as the key exchange, hashing and signing algorithms, respectively. It also generates the endorsement keys (EK), using the TSS function `tpm2_createek`. The keys are stored inside the TPM's persistent secure storage using the TSS function `tpm2_evictcontrol`. This process is performed after booting and also at each attestation interval.

When the attestation timeout is reached, GW_X transmits the EK and AK public keys to GW_Y . The latter uses the EK public key in order to encrypt the challenge, so that only that exact TPM can decrypt it, using the TSS function `tpm2_makecredential`. To perform the attestation, GW_X decrypts the challenge with the EK private key and uses the TSS function `tpm2_quote` with the AK private key to hash and sign the attested values. Finally, GW_Y verifies the signature using the TSS `tpm2_verifysignature` with the AK public key, before validating the attestation report.

IoT Devices: We implemented the IoT functionality using the Pycom Pysense and FiPy devices.⁶ Pysense is a small, low-power board that includes sensors for measuring temperature, humidity, pressure, and light, as well as accelerometer, magnetometer, and gyroscope. FiPy, is attached to the Pysense board and offers a range of connectivity options, including Wi-Fi, Bluetooth, LoRa, Sigfox, and NB-IoT. FiPy board incorporates an ESP32 (Xtensa dual-core 32-bit LX6) microprocessor running at 160 MHz with 520 KB on-chip SRAM, 4MB external pSRAM and 8MB flash.

IoT PUF Mechanism: To integrate the PUF mechanism in our IoT testbed, we chose a static random access memory (SRAM) PUF; SRAM PUFs rely on the random variations in the power-up contents of an SRAM array to generate unique and unclonable cryptographic keys [36]. They are widely used in constrained IoT devices since they do not require extra resources (most IoT devices are equipped with a microcontroller with an on-chip SRAM) and impose negligible computing and energy costs [37].

The ESP32 SRAM of the Pycom devices deployed in our testbed can be used for PUF key generation. Ashur et al. [38] tested 128 KB on-chip SRAM cells of a similar Pycom device (LoPy) to generate 100 128-bit PUF keys and showed that this SRAM PUF scheme has good PUF reliability and uniqueness. To conduct a similar SRAM PUF feasibility study, we experimented with the ESP-WROVER-KIT V4.1 development board. This board is equipped with an ESP32 uC and provides utilities that assisted us in building a custom bootloader, which does not interfere with the initialization of on-chip SRAM and reads out its power-up values. We powered up the board several times to check for memory cells in the on-chip SRAM that have stable power-up values and showed that we can establish a stability map consisting of several hundreds of bytes similar to [38].

B. Performance and Energy Consumption Analysis

To analyze the computation, network and energy consumption overheads imposed on the IoT devices and the gateways by the execution of the continuous authentication and attestation mechanisms, we implemented the HA-CAAP functions on our testbed devices (Pycom Pysense/FiPy and Raspberry Pi) and measured their performance.

1) Use Case: We assume a food supply chain (e.g., fruits), in which product quality, appearance and nutritional value can be affected by environmental parameters. We assume that the products are packaged in boxes of 5 Kgr and each box

⁵<https://hyperledger.github.io/caliper/>

⁶<https://pycom.io/>

TABLE III
IoT DEVICE NORMAL AND AUTHENTICATION FUNCTIONS WORKLOAD

Workload	Execution time (ms)
Acquisition (temperature, humidity, pressure, light)	$521 + 315 + 520 + 33 = 1389$
Processing (average, out-of-limit detection)	3.38
Authentication (GenPUF, GenDevKey, GenResp)	$0.1 + 3 + 5 = 8.1$ (Overhead: 0.6%)

is monitored by a set of four sensors: temperature, relative humidity, light, and pressure. The payload length of each sensor is 32 bytes, including the sensor value, timestamp, sensor ID, and other contextual data. For our analysis, we consider different parameters for the sensor network dimensions and the data acquisition period. We assume that: 1) the supply chain tracking system contains 10, 100, or 200 sets of sensors to monitor 50 Kgr, 500 Kgr, or 1 tone of foods, respectively. The fruit is packaged in boxes of 5 Kgr and each box contains one sensor set and 2) the sensors collect data every 30 s, 1 min, 5 min, or 10 min depending on the environmental conditions.

2) *IoT Device Performance Analysis*: We analyze the continuous authentication overhead in the IoT devices from three perspectives: 1) the extra computing workload; 2) the communication overhead in the BLE network due to the transmission of protocol data; and 3) the extra energy consumed.

IoT Devices Sensor Operations: A typical flow of IoT sensing devices is to: wake up periodically from sleep mode, read and process sensor data, activate the radio (e.g., BLE) and transmit data to the gateway, and go again to sleep.

For our testbed, we implemented a simple workload scenario in the Pycom/PySense devices that periodically reads four sensors (temperature, humidity, light and pressure), and performs two basic operations: 1) an averaging function and 2) an out-of-limit detection of the sensor values. The data acquisition time for the four sensors and the execution times of the data processing functions are presented in Table III.

IoT Devices Authentication Functions: Their basic building blocks are: 1) GenPUF generates the PUF response based on the received challenge using the SRAM PUF data stored in the nonvolatile on-chip memory; 2) GenDevKey implements the Hash function into the output of the GenPUF to generate the secure device key; and 3) GenResp implements the hash-based message authentication code (HMAC) and the advanced encryption standard (AES) algorithm to generate the final PUF response. The execution time of the authentication functions, when executed in the Pycom FiPy device, are shown in Table III. Our IoT devices necessitate roughly 8.1 milliseconds in total to execute these functions imposing a negligible overhead (less than 0.6%) in the typical workload.

To evaluate the communication overhead, we calculate the average time required for N devices to connect and transmit the sensor and the protocol data to the gateway through a BLE network. The time needed for each device to transmit a message is calculated as

$$\frac{S_{\text{data}} + S_{\text{ack}}}{\text{BDR}} \cdot N_p \cdot (1 + N_{\text{retr}}) + T_{\text{conn-int}} + T_{\text{ch-hop}} + T_{\text{adv-scan}}$$

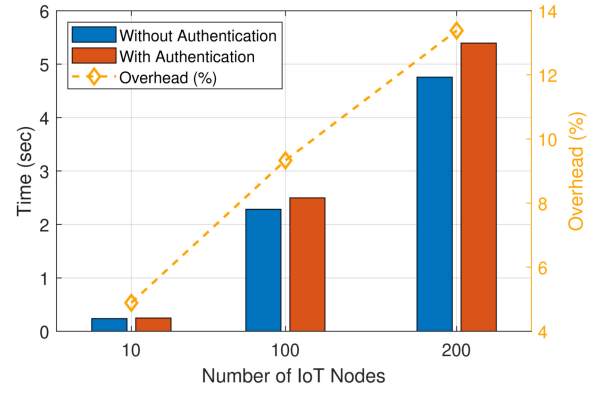


Fig. 8. BLE communication overhead.

where, S_{data} and S_{ack} define the data and acknowledgment packet size (in bits), respectively, including the payload, the header and the CRC field, BDR represents the BLE data rate, N_p represents the number of packets per message, N_{retr} is the number of retransmissions to handle collisions, and $T_{\text{conn-int}}$, $T_{\text{ch-hop}}$, and $T_{\text{adv-scan}}$ are typical BLE timing parameters, which denote the connection interval, the channel hopping time, and the time needed for advertising or scanning. In our use case, we assume that the data message (i.e., sensing data) size is 128 bytes, and with the protocol overhead is increased to 256. The following BLE parameters are also assumed: $S_{\text{data}} = 256$ bytes, $S_{\text{ack}} = 3$ bytes, BDR = 1 Mb/s, $T_{\text{conn-int}}$, $T_{\text{ch-hop}}$ and $T_{\text{adv-scan}}$ are 30, 1, and 10 ms, respectively and N_{retr} is 0, 1 and 2 for 10, 100, and 200 devices, respectively. Finally, we assume that the average number of attempts for an IoT device to gain access to the BLE network is $(N+1)/2$.

Fig. 8 presents the transmission delay of an IoT device in the BLE network without the authentication functions (i.e., the message includes only the sensor data) and with the authentication functions (i.e., the payload includes the sensor data plus the protocol data). The communication overhead of the protocol in the BLE network is less than 5% for 10 IoT devices and scales at around 13% at 200 devices.

Finally, we estimate the energy consumption overhead of the IoT devices to support the authentication protocol, which includes the extra energy consumed by the microcontroller to execute the protocol functions (see Table III) and the extra energy to transmit the protocol data to the BLE network (see Fig. 8). The energy consumption both with and without the continuous authentication support for 10, 100, and 200 IoT devices, and different communication intervals, is shown in Fig. 9. The Pycom FiPy device has various power states that can help reduce power consumption. In idle operating mode (all radio communications disabled), it consumes 62.7 mA at 5V, while with BLE enabled, the average power consumption is almost double (121 mA). In addition, power consumption in deep sleep mode is significantly reduced to 0.25–0.35 mA.

Dotted curves, in Fig. 9, indicate the energy with continuous authentication support and rounded curves without it. The analysis reveals that as the communication interval increases (i.e., HA-CAAP runs less frequently), the overhead diminishes and approaches insignificance. Conversely, the overhead escalates with an increasing number of devices. The most critical

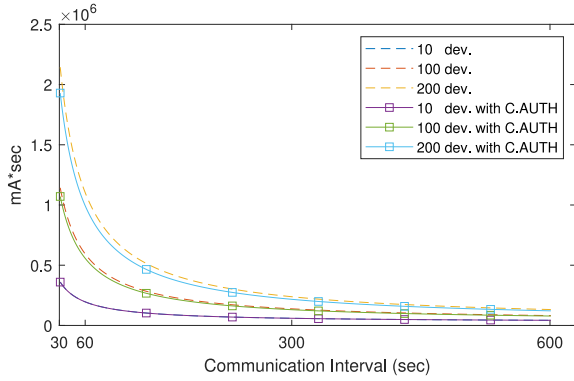


Fig. 9. Energy consumption of IoT devices with and without continuous authentication.

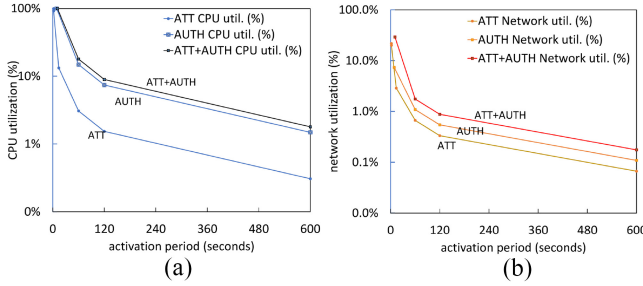


Fig. 10. Continuous attestation and authentication cost in (a) gateway CPU utilization and (b) in network utilization.

scenario occurs in 200 nodes with a communication interval of 30 s, resulting in an 11% energy consumption increase.

3) *Gateway Performance Analysis*: We analyze the performance of gateways from three perspectives: 1) the performance of the gateways' cryptographic operations; 2) the CPU utilization; and 3) the network utilization.

Benchmarking of Gateway Cryptographic Operations: The gateway sign and verify operations on the RoT module require $t_{\text{sign}} = 0.6$ and $t_{\text{verify}} = 0.34$ s, respectively. The size s_{ar} of the attestation report is $s_{ar} = 500$ bytes. Therefore, the blockchain synchronization time t_{bs} for the attestation report is given by: $t_{bs} = t_w \cdot \lceil s_{ar}/100 \rceil$, which, in the case at hand, is evaluated to $t_{bs} = 0.9$ s, as 5 writing operations will be required on the blockchain. The total CPU time t_{att} on the gateway side for the attestation protocol (Fig. 4) is given by: $t_{att} = t_{\text{sign}} + t_{bs} + t_{\text{verify}} = 1.84$ s.

CPU Utilization Tradeoffs of Proposed Protocol: To evaluate the tradeoffs between the activation period of the attestation protocol (T_{att}) and the CPU utilization of the gateways U_{CPU} , we consider the following formula: $U_{CPU}(t_{att}) = t_{att}/T_{att} \cdot 100$. Note that we consider the CPU of the gateway to remain occupied in a blocking assignment state, while the blockchain synchronizes data during a write operation. Similarly, to evaluate the tradeoffs between the activation period of the authentication protocol (T_{auth}) and the CPU utilization of the gateways, we evaluate the time t_{auth} required to execute the authentication protocol (Fig. 3) on a gateway. The total time for t_{auth} is experimentally evaluated to 8.9 s from which 6.04 s are required to verify the responses of the devices and 1.65 s for the operation *publishInADL()*. Note

that these numbers are obtained considering the corner-case scenario of $|D| = 200$ IoT devices and $|G| = 200$ gateways. The gateways' CPU utilization as a function of the activation period for the attestation and authentication subprotocols of the proposed protocol are depicted separately (labeled as ATT and AUTH, respectively) and together (labeled as ATT+AUTH) in Fig. 10(a). An attestation period of $T_{att} = 1.84$ s leads to 100% CPU utilization. However, CPU utilization drops significantly during slightly slower activation periods. When the activation period of the attestation protocol is one minute, ($T_{att} = 60$) s, the CPU utilization is only 3.07%, and drops below 1% for attestation period $T_{att} \geq 5$ minutes. The CPU utilization cost of the authentication protocol is higher at the 60-s activation period (14.8%), however, it drops below 7.4% for 2 min. For both authentication and attestation protocols running, they have less than 8.9% CPU utilization cost on the gateway side. Note that this cost is evaluated only on a single core of the gateway, while the additional cores on the gateway do not perform any duty. Therefore, we conclude that the CPU utilization for the proposed protocol is low.

Network Utilization Tradeoffs of Proposed Protocol: The proposed protocol has also network throughput implications. The network cost of a gateway when executing a workload with a period T that transmits S bytes of data is evaluated using $C_{Net}(T, S) = (S/T) \cdot (8/1000000)$, and it is expressed in Mb/s. The network throughput utilization of the attestation protocol U_{Net} is evaluated using: $U_{Net}(T_{att}) = C_{Net}(T_{att}, s_{ar})/N_{TP}$, where s_{ar} the attestation report size, and $N_{TP} = 2$ Mb/s is the maximum throughput of the considered network. Similarly, the network cost of the authentication protocol is evaluated using $U_{Net}(T_{auth}) = C_{Net}(T_{auth}, s_{gr})/N_{TP}$, where $s_{gr} = 4 \cdot |D| + 16$ is the number of bytes for publishing the groups of $|D|$ number of devices in the blockchain. For $|D| = 20$ devices, s_{gr} is evaluated to 816 bytes. U_{Net} is plotted in Fig. 10(b). We observe that the network throughput utilization of the attestation protocol is 20% for an activation period $T_{att} = 2$ s. However, it drops below 1% for $T_{att} \geq 30$ s. Slightly higher is the network utilization cost of the authentication protocol, which drops below 1% for $T_{auth} \geq 60$ s. When both protocols are activated, the network throughput of a 2 Mb/s network is affected by less than 1% for an activation period of 120 s. Note that T_{att} and T_{auth} can be configured to different values to minimize further their impact.

4) *Scalability for Multiauthority Settings*: We examine the scalability of HA-CAAP for multiauthority settings. In the examined scenario, a food supply chain may involve multiple authorities from different sectors, such as producers, transporters, and sales points. Assume that each producer is the owner of the IoT devices installed in its products' delivery boxes. In the simplest scenario, if each device owner trusts all other authorities, then all entries containing the devices' fingerprints in the *EDL* list will be encrypted with a key shared among all the authorities, and the efficiency of the protocol is not affected in any way. In practice, however, more complex scenarios may arise. For example, a producer may want to share the fingerprints of its IoT devices only with some transporters and sale points. In that case, for granular access to device information, each producer will encrypt the

entries for its IoT devices, using a different group key shared only with its trusted subgroup of authorities. Note that this extra encryption cost only affects the device enrollment phase, is performed once, and is linear to the number of trusted authorities' subgroups. Besides this, all the communication, computation, and energy costs for HA-CAAP execution are not affected.

VI. CONCLUSION

Existing continuous IoT authentication mechanisms are targeted to environments where the gateways are managed by a single authority and/or the IoT devices remain connected at the same network domain. In multiauthority and multidomain environments, however, there is a need to distributedly manage gateways' trust and IoT devices' authenticity, in real-time. HA-CAAP is a protocol specifically targeted to such dynamic environments. At the gateways' side, HA-CAAP combines the hardware immutability of TEE with the software immutability and synchronization of blockchain technology, to achieve trusted execution without relying on strong trust assumptions. On the IoT devices' side, continuous authentication and key agreement rely on a flexible PUF-based mechanism. Our testbed implementation verifies that HA-CAAP can be efficiently implemented in realistic scenarios with reasonable hardware assumptions for both gateways and IoT devices.

REFERENCES

- [1] T. Yaqoob, H. Abbas, and M. Atiquzzaman, "Security vulnerabilities, attacks, countermeasures, and regulations of networked medical devices—A review," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 4, pp. 3723–3768, 4th Quart., 2019.
- [2] Y.-H. Chuang, N.-W. Lo, C.-Y. Yang, and S.-W. Tang, "A lightweight continuous authentication protocol for the Internet of Things," *Sensors*, vol. 18, no. 4, p. 1104, 2018.
- [3] S. W. Shah, N. F. Syed, A. Shaghghi, A. Anwar, Z. Baig, and R. Doss, "LCDA: Lightweight continuous device-to-device authentication for a zero trust architecture (ZTA)," *Comput. Security*, vol. 108, Sep. 2021, Art. no. 102351.
- [4] A. Badhib, S. Alshehri, and A. Cherif, "A robust device-to-device continuous authentication protocol for the Internet of Things," *IEEE Access*, vol. 9, pp. 124768–124792, 2021.
- [5] B. Yu, C. Yang, and J. Ma, "Continuous authentication for the Internet of Things using channel state information," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, 2019, pp. 1–6.
- [6] S. Alshehri and O. Bamasag, "AAC-IoT: Attribute access control scheme for IoT using lightweight cryptography and hyperledger fabric blockchain," *Appl. Sci.*, vol. 12, no. 16, p. 8111, 2022.
- [7] F. H. Al-Naji and R. Zagrouba, "CAB-IoT: Continuous authentication architecture based on blockchain for Internet of Things," *J. King Saud Univ.-Comput. Inf. Sci.*, vol. 34, no. 6, pp. 2497–2514, 2022.
- [8] R. Sarabi Miyanaaji, S. Jabbehdari, and N. Modiri, "Continuous lightweight authentication according group priority and key agreement for Internet of Things," *Trans. Emerg. Telecommun. Technol.*, vol. 33, no. 7, 2022, Art. no. e4479.
- [9] H. Birkholz, D. Thaler, M. Richardson, N. Smith, and W. Pan, "Remote attestation procedures (RATS) architecture," Internet Eng. Task Force, RFC 9334, 2023.
- [10] S. Johnson, V. Scarlata, C. Rozas, E. Brickell, and F. Mckeen, "Intel software guard extensions: Epid provisioning and attestation services," Intel Corp., Santa Clara, CA, USA, White Paper, 2016.
- [11] S. Pinto and N. Santos, "Demystifying arm trustzone: A comprehensive survey," *ACM Comput. Surv.*, vol. 51, no. 6, pp. 1–36, 2019.
- [12] (Arm, Cambridge, U.K.). *Arm Security Manifesto*. Jun. 2021. [Online]. Available: <https://interactive.arm.com/story/security-manifesto-2021/>
- [13] S. P. Mohanty, V. P. Yanambaka, E. Kougianos, and D. Puthal, "PUFchain: A hardware-assisted blockchain for sustainable simultaneous device and data security in the Internet of Everything (IoE)," *IEEE Consum. Electron. Mag.*, vol. 9, no. 2, pp. 8–16, Mar. 2020.
- [14] F. Amsaad and S. Köse, "A secure lightweight hardware-assisted charging coordination authentication framework for trusted smart grid energy storage units," *SN Comput. Sci.*, vol. 2, no. 6, pp. 1–15, 2021.
- [15] P. Mall, R. Amin, A. K. Das, M. T. Leung, and K.-K. R. Choo, "PUF-based authentication and key agreement protocols for IoT, WSNs and smart grids: A comprehensive survey," *IEEE Internet Things J.*, vol. 9, no. 11, pp. 8205–8228, Jun. 2022.
- [16] W. Liang, S. Xie, J. Long, K.-C. Li, D. Zhang, and K. Li, "A double PUF-based RFID identity authentication protocol in service-centric internet of things environments," *Inf. Sci.*, vol. 503, pp. 129–147, Nov. 2019.
- [17] M. Ebrahimabadi, M. Younis, and N. Karimi, "A PUF-based modeling-attack resilient authentication protocol for IoT devices," *IEEE Internet Things J.*, vol. 9, no. 5, pp. 3684–3703, Mar. 2022.
- [18] K. Lounis and M. Zulkernine, "T2T-MAP: A PUF-based thing-to-thing mutual authentication protocol for IoT," *IEEE Access*, vol. 9, pp. 137384–137405, 2021.
- [19] V. K. Bathalapalli, S. P. Mohanty, E. Kougianos, B. K. Baniya, and B. Rout, "PUFchain 3.0: Hardware-assisted distributed ledger for robust authentication in the Internet of Medical Things," in *Proc. IFIP Int. Internet Things Conf.*, 2022, pp. 23–40.
- [20] N. Yu, J. Ma, X. Jin, J. Wang, and K. Chen, "Context-aware continuous authentication and dynamic device pairing for enterprise IoT," in *Proc. Int. Conf. Internet Things*, 2019, pp. 114–122.
- [21] L. Meng, D. Huang, J. An, X. Zhou, and F. Lin, "A continuous authentication protocol without trust authority for zero trust architecture," *China Commun.*, vol. 19, no. 8, pp. 198–213, Aug. 2022.
- [22] A. O. Gomez Rivera, D. K. Tosh, and U. Ghosh, "Resilient sensor authentication in SCADA by integrating physical unclonable function and blockchain," *Clust. Comput.*, vol. 25, no. 3, pp. 1869–1883, 2022.
- [23] K. Goutsos and A. Bystrov, "Lightweight PUF-based continuous authentication protocol," in *Proc. Int. Conf. Comput., Electron., Commun. Eng. (ICCECE)*, 2019, pp. 229–234.
- [24] S. Alshomrani and S. Li, "PUFDCA: A zero-trust-based IoT device continuous authentication protocol," *Wireless Commun. Mobile Comput.*, vol. 2022, no. 1, 2022, Art. no. 6367579, doi: [10.1155/2022/6367579](https://doi.org/10.1155/2022/6367579).
- [25] D. Li et al., "Blockchain-based authentication for IIoT devices with PUF," *J. Syst. Archit.*, vol. 130, Sep. 2022, Art. no. 102638.
- [26] K. Qian, Y. Liu, X. He, M. Du, S. Zhang, and K. Wang, "HPCchain: A consortium blockchain system based on CPU-FPGA hybrid PUF for Industrial Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 19, no. 11, pp. 11205–11215, Nov. 2023.
- [27] W. Wang et al., "Blockchain and PUF-based lightweight authentication protocol for wireless medical sensor networks," *IEEE Internet Things J.*, vol. 9, no. 11, pp. 8883–8891, Jun. 2022.
- [28] A. Ibrahim, A.-R. Sadeghi, and G. Tsudik, "AID: Autonomous attestation of IoT devices," in *Proc. SRDS*, 2018, pp. 1–10.
- [29] M. Conti, P. Kaliyar, M. M. Rabbani, and S. Ranise, "Attestation-enabled secure and scalable routing protocol for IoT networks," *Ad Hoc Netw.*, vol. 98, Mar. 2020, Art. no. 1020540.
- [30] E. Dushku, M. M. Rabbani, M. Conti, L. V. Mancini, and S. Ranise, "SARA: Secure asynchronous remote attestation for IoT systems," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 3123–3136, 2020.
- [31] L. Moreau, E. Conchon, and D. Sauveron, "CRAFT: A continuous remote attestation framework for IoT," *IEEE Access*, vol. 9, pp. 46430–46447, 2021.
- [32] D. Paulin, C. Hennebert, T. Franco-Rondisson, R. Jayles, T. Loubier, and R. Collado, "HistoTrust: Ethereum-based attestation of a data history built with OP-TEE and TPM," in *Proc. Int. Symp. Found. Pract. Security*, 2021, pp. 130–145.
- [33] S. F. J. J. Ankergård, E. Dushku, and N. Dragoni, "PERMANENT: Publicly verifiable remote attestation for Internet of Things through blockchain," in *Proc. Int. Symp. Found. Pract. Security*, 2021, pp. 218–234.
- [34] C. Xu, H. Liu, P. Li, and P. Wang, "A remote attestation security model based on privacy-preserving blockchain for V2X," *IEEE Access*, vol. 6, pp. 67809–67818, 2018.
- [35] (Infineon, Neubiberg, Germany). *Setup and Operation Guide for Optiga TPM Backed Remote Attestation on a Linux System*. 2020. [Online]. Available: <https://github.com/Infineon/remot-attestation-optiga-tpm>
- [36] D. E. Holcomb, W. P. Burleson, and K. Fu, "Power-up SRAM state as an identifying fingerprint and source of true random numbers," *IEEE Trans. Comput.*, vol. 58, no. 9, pp. 1198–1210, Sep. 2009.

- [37] F. Farha, H. Ning, K. Ali, L. Chen, and C. Nugent, "SRAM-PUF-based entities authentication scheme for resource-constrained IoT devices," *IEEE Internet Things J.*, vol. 8, no. 7, pp. 5904–5913, Apr. 2021.
- [38] T. Ashur et al., "A privacy-preserving device tracking system using a low-power wide-area network," in *Proc. Int. Conf. Cryptol. Netw. Security*, 2018, pp. 347–369.

Vangelis Malamas (Member, IEEE) received the first B.Sc. degree in mathematics from the University of Patras, Patras, Greece, in 2010, the M.Sc. degree in computer science from the University of Piraeus, Piraeus, Greece, in 2016, the second B.Sc. degree in computer science from Hellenic Open University, Patra, Greece, in 2021, and the Ph.D. degree in distributed security and trust management technologies for IoT from the University of Piraeus in 2024.

He is a Postdoctoral Researcher with the University of Piraeus and a Senior Consultant with the Greek Ministry of Digital Governance. He has contributed to numerous EU and national research projects and authored several publications in international conferences and journals. His research is focused on blockchain security, Metaverse, AI, and digital identities.

Panayiotis Kotzanikolaou (Member, IEEE) received the B.Sc. degree in computer science and the Ph.D. degree in ICT security from the University of Piraeus, Piraeus, Greece, in 1998 and 2003, respectively.

He is a Professor of Network Security and Privacy with the Department of Informatics, University of Piraeus. He has worked in the field of cybersecurity for more than 20 years both in the private and public sector and has participated in various research and development projects. He has published more than 100 papers in books, journals, and international conferences in the area of cybersecurity. He has received various certifications in cybersecurity.

Kostas Nomikos received the B.Sc. degree in informatics and the M.Sc. degree from the University of Piraeus, Piraeus, Greece, in 2015 and 2019, respectively, where he is currently pursuing the Ph.D. degree.

He has extensive experience in embedded security, particularly in the areas of machine learning attacks and side channel attacks, and was an Embedded Software Engineer. His work spans involvement in European and national research projects, where he applies cutting-edge technologies to enhance system security and resilience. His research focuses on countermeasures against machine learning attacks utilizing side channel leakages.

Christos Zonios received the B.Sc. degree in computer science and the M.Sc. degree in data and computing systems engineering from the Department of Computer Science and Engineering, University of Ioannina, Ioannina, Greece, in 2020 and 2021, respectively, where he is currently pursuing the Ph.D. degree in security and reliability of neuromorphic computing systems.

He is currently a Research Associate with the Centre for Research and Technology Hellas, Marousi, Greece. He has participated in multiple security and AI-oriented projects funded by the EC and the Greek Government. His research focuses on AI and systems security, power side-channel analysis, large language models, anomaly detection, confidential computing, and AI IoT systems.

Vasileios Tenentes (Member, IEEE), received the B.Sc. degree in computer science from the University of Piraeus, Piraeus, Greece, in 2003, and the M.Sc. and Ph.D. degrees in computer science and engineering from the University of Ioannina, Ioannina, Greece, in 2007 and 2013, respectively.

He is currently an Associate Professor of Digital Systems with the University of Ioannina, and participates with the Hellenic Chips Competence Centers Program under the ChipsJU Action of EU. He has published over 44 papers in international journals and conferences, and holds 3 patents on hardware and systems security. His research interests include electronic design automation, hardware design for security, VLSI design and test, embedded systems and IoT, emerging VLSI technologies, and post-von Neumann computing architectures for emerging applications.

Dr. Tenentes is a member of HiPEAC in 2023.

Mihalis Psarakis (Member, IEEE) received the Diploma degree in computer engineering from the Department of Computer Engineering and Informatics, University of Patras, Patras, Greece, in 1995, and the Ph.D. degree in computer science from the Department of Informatics and Telecommunications, University of Athens, Athens, Greece, in 2001.

He is currently an Associate Professor with the Department of Informatics, University of Piraeus, Piraeus, Greece, and also the Director of the Embedded Systems Laboratory. He has participated in and coordinated several research and development projects financed by EU, ESA, and national funds. He has published more than 100 papers in peer-reviewed transactions, journals, and conference proceedings. His research interests include the design of secure and reliable embedded systems using microprocessors, systems-on-chip, and commercial-off-the-self field-programmable gate arrays.